Breaking Changes in R: A Case for Reproducible Research

Why Package Version Management Matters

PHB 243b: Practicum in Biostatistics

2026-01-13

Contents

1 Introduction

Reproducibility is a cornerstone of scientific research. In computational research, reproducibility requires not only access to the original code and data but also the exact computational environment in which the analysis was conducted. This includes the versions of R and all packages used. Unfortunately, package updates—even minor ones—can introduce *breaking changes* that silently alter results or cause code to fail entirely.

This white paper presents six real-world examples of breaking changes in widely-used R packages. Each example demonstrates how code that worked correctly at one point in time may produce different results or errors when run with a newer package version. These examples provide compelling motivation for using environment management tools like `renv` and containerization with Docker.

2 Example 1: The `stringsAsFactors` Default Change (R 4.0.0)

2.1 The Change

R 4.0.0, released in April 2020, changed the default value of `stringsAsFactors` from `TRUE` to `FALSE` in `data.frame()` and `read.table()` family functions. This was one of the most significant breaking changes in R's history.

2.2 Historical Context

- **R 0.62 (1998)**: `data.frame()` always converted strings to factors
- **R 2.4.0 (2006)**: Added `stringsAsFactors` argument, defaulting to `TRUE`
- **R 4.0.0 (2020)**: Changed default to `FALSE`

2.3 Impact Example

```r
# Sample clinical trial data
patient_data <- data.frame(
  patient_id = c("PT001", "PT002", "PT003"),
  treatment = c("Drug A", "Placebo", "Drug A"),
  response = c(1.2, 0.8, 1.5)
)

# Pre-R 4.0 behavior: treatment is automatically a factor
# Post-R 4.0 behavior: treatment remains character

# Code that worked before R 4.0.0:
model <- lm(response ~ treatment, data = patient_data)
# Pre-R 4.0: Works correctly with factor predictor
# Post-R 4.0: Works but treats "treatment" as character,
#             potentially yielding different coefficient interpretations

# More critically, this code fails silently:
levels(patient_data$treatment)
# Pre-R 4.0: c("Drug A", "Placebo")
# Post-R 4.0: NULL (no error, just NULL)
```

2.4 The Reproducibility Problem

Consider a researcher who published an analysis in 2019 using R 3.6:

```r
# Original analysis (2019, R 3.6.x)
clinical_data <- read.csv("trial_results.csv")

# Researcher assumes treatment column is factor (it was, automatically)
summary_by_arm <- clinical_data |>
  split(clinical_data$treatment) |>
  lapply(function(x) mean(x$response))

# Factor ordering determined presentation order in results
# In 2025 with R 4.4.x: character ordering (alphabetical) is used instead
```

2.5 Defensive Solution

```r
# Explicit factor conversion (version-independent)
patient_data <- data.frame(
  patient_id = c("PT001", "PT002", "PT003"),
  treatment = factor(c("Drug A", "Placebo", "Drug A")),
  response = c(1.2, 0.8, 1.5)
)

# Or use readr::read_csv() with explicit col_types
library(readr)
clinical_data <- read_csv(
  "trial_results.csv",
  col_types = cols(treatment = col_factor())
)
```

3 Example 2: `dplyr::arrange()` Locale Change (dplyr 1.1.0)

3.1 The Change

Starting with dplyr 1.1.0 (January 2023), `arrange()` and `group_by()` default to using the "C" locale for sorting character vectors, rather than the system's locale.

3.2 Impact Example

```
library(dplyr)

# International researcher names
researchers <- tibble(
  name = c("Müller", "Martinez", "Mäkinen", "Morris", "Møller"),
  publications = c(45, 32, 28, 51, 39)
)

# Pre-dplyr 1.1.0: Sort order depended on system locale
# - German locale: Müller, Mäkinen, Martinez, Morris, Møller
# - US English: Martinez, Morris, Mäkinen, Møller, Müller
# - Danish locale: Martinez, Morris, Müller, Mäkinen, Møller

# Post-dplyr 1.1.0: C locale (ASCII-based) ordering
researchers |>
  arrange(name)
# Always returns: Martinez, Morris, Mäkinen, Müller, Møller
```

3.3 The Reproducibility Problem

```
# Researcher A (2022, French system locale, dplyr 1.0.x)
top_3 <- researchers |>
  arrange(name) |>
  head(3) |>
  pull(name)
# Result: locale-dependent selection

# Researcher B (2025, dplyr 1.1.4)
top_3_new <- researchers |>
  arrange(name) |>
  head(3) |>
  pull(name)
# Result: C locale ordering

identical(top_3, top_3_new)  # FALSE - different subgroup selected!
```

3.4 Defensive Solution

```
# Explicit locale specification
researchers |>
  arrange(name, .locale = "en")

# Or restore legacy behavior (not recommended for new code)
options(dplyr.legacy_locale = TRUE)
```

4 Example 3: ggplot2 Size to Linewidth (ggplot2 3.4.0)

4.1 The Change

ggplot2 3.4.0 (November 2022) introduced a new `linewidth` aesthetic, separating line thickness from point size. The `size` aesthetic for lines was deprecated.

4.2 Impact Example

```
library(ggplot2)

# Data for plotting
df <- data.frame(
  x = 1:10,
  y = cumsum(rnorm(10))
)

# Pre-ggplot2 3.4.0 code:
ggplot(df, aes(x, y)) +
  geom_line(size = 2)  # Worked without warning

# Post-ggplot2 3.4.0:
# Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
# Please use `linewidth` instead.

# The plot still renders, but with a deprecation warning that may
# become an error in future versions
```

4.3 Silent Breaking Change with `geom_pointrange()`

```
# This is more insidious - no warning, but behavior changes
summary_data <- data.frame(
  group = c("A", "B", "C"),
  mean = c(2.5, 3.1, 2.8),
  lower = c(2.0, 2.6, 2.3),
  upper = c(3.0, 3.6, 3.3)
)

# Pre-3.4.0: size controls BOTH point and line thickness
ggplot(summary_data, aes(group, mean, ymin = lower, ymax = upper)) +
  geom_pointrange(size = 1.5)

# Post-3.4.0: size controls ONLY point size, lines use default width
# No warning issued! Visual output silently changes.
```

4.4 Defensive Solution

```
# Explicit specification of both aesthetics
ggplot(summary_data, aes(group, mean, ymin = lower, ymax = upper)) +
  geom_pointrange(size = 1.5, linewidth = 1.5)

# For simple line plots
```

```r
ggplot(df, aes(x, y)) +
  geom_line(linewidth = 2)
```

5 Example 4: `readr` Column Type Guessing Changes

5.1 The Change

readr 1.2.0 changed the column type guessing behavior: columns that would previously be guessed as integer are now guessed as numeric (double). The `guess_max` parameter behavior has also evolved across versions.

5.2 Impact Example

```r
library(readr)

# Consider a CSV where the first 1000 rows have integer-like values
# but later rows have decimals

# File contents (simulated):
# id,measurement
# 1,100
# 2,200
# ... (rows 3-1000 are integers)
# 1001,150.5  # First decimal value

# Pre-readr 1.2.0 behavior:
# measurement column guessed as integer from first 1000 rows
# Row 1001 causes parsing failure or coercion issues

# Post-readr 1.2.0 behavior:
# measurement column guessed as double (numeric)
# All rows parse correctly
```

5.3 Real-World Data Issue

```r
# Biomedical data with ID column
lab_results <- read_csv("lab_data.csv")

# Pre-1.2.0: patient_id might be guessed as integer
# If patient IDs later include alphanumeric codes, parsing fails

# The guess_max default (1000 rows) can miss patterns:
large_dataset <- read_csv("longitudinal_study.csv")
# First 1000 rows: all measurements are whole numbers
# Rows 5000+: decimal values appear
# Column type mismatch causes silent coercion or errors
```

5.4 Defensive Solution

```r
# Always specify column types explicitly
lab_results <- read_csv(
  "lab_data.csv",
  col_types = cols(
    patient_id = col_character(),
    measurement = col_double(),
    visit_date = col_date(format = "%Y-%m-%d")
  )
```

```
)

# Or increase guess_max for large files (second edition parser)
large_dataset <- read_csv(
  "longitudinal_study.csv",
  guess_max = Inf  # Safe with readr 2.0+ second edition parser
)
```

6 Example 5: `tidyr` Pivot Functions Replace gather/spread

6.1 The Change

tidyr 1.0.0 (September 2019) introduced `pivot_longer()` and `pivot_wider()` as replacements for `gather()` and `spread()`. The older functions are now in "superseded" lifecycle status.

6.2 Impact Example

```
library(tidyr)

# Wide-format clinical data
wide_data <- data.frame(
  patient = c("P1", "P2", "P3"),
  visit_1 = c(120, 135, 128),
  visit_2 = c(118, 140, 125),
  visit_3 = c(122, 138, 130)
)

# Old approach (still works but superseded):
long_old <- gather(wide_data, visit, bp, visit_1:visit_3)

# New approach:
long_new <- pivot_longer(
  wide_data,
  cols = visit_1:visit_3,
  names_to = "visit",
  values_to = "bp"
)

# Results are equivalent, but gather() receives no new features
# and may eventually show deprecation warnings
```

6.3 Breaking Change: Argument Position

```
# The ... argument position changed in pivot functions
# Code relying on positional arguments may break

# Additionally, nest() and unnest() behaviors changed significantly
# between tidyr 0.8.x and 1.0.0

# Old nest syntax (tidyr < 1.0.0):
# nested <- df |> nest(-group_var)

# New nest syntax (tidyr >= 1.0.0):
# nested <- df |> nest(data = c(col1, col2, col3))
# or
# nested <- df |> nest(.by = group_var)
```

6.4 Defensive Solution

```
# Use named arguments explicitly
long_data <- pivot_longer(
```

```
  data = wide_data,
  cols = starts_with("visit"),
  names_to = "visit",
  values_to = "bp",
  names_prefix = "visit_"
)

# Migrate from gather/spread to pivot functions for new code
# Lock tidyr version with renv for legacy code
```

7 Example 6: `dplyr::summarise()` Grouping Behavior

7.1 The Change

dplyr 1.0.0 (May 2020) changed how `summarise()` handles grouping structure after aggregation. Additionally, dplyr 1.1.0 deprecated returning more or fewer than one row per group from `summarise()`, introducing `reframe()` as the replacement.

7.2 Impact Example

```
library(dplyr)

# Multi-level grouping
clinical_trial <- tibble(
  site = rep(c("Hospital A", "Hospital B"), each = 6),
  arm = rep(c("Treatment", "Control"), 6),
  patient = 1:12,
  response = rnorm(12, mean = 10, sd = 2)
)

# Summarize by site and arm
summary_stats <- clinical_trial |>
  group_by(site, arm) |>
  summarise(mean_response = mean(response))

# Pre-dplyr 1.0.0: Result was ungrouped
# Post-dplyr 1.0.0: Result is grouped by 'site' (drops last group)
# This message appears:
# `summarise()` has grouped output by 'site'. You can override using
# the `.groups` argument.
```

7.3 The `reframe()` Requirement

```
# Code that returned multiple rows per group:
# Pre-dplyr 1.1.0: Worked with warning
quantile_summary <- clinical_trial |>
  group_by(arm) |>
  summarise(
    quantile = c(0.25, 0.50, 0.75),
    value = quantile(response, c(0.25, 0.50, 0.75))
  )
# Warning: Returning more (or less) than 1 row per `summarise()` group
# was deprecated in dplyr 1.1.0.

# Post-dplyr 1.1.0: Must use reframe()
quantile_summary <- clinical_trial |>
  group_by(arm) |>
  reframe(
    quantile = c(0.25, 0.50, 0.75),
    value = quantile(response, c(0.25, 0.50, 0.75))
  )
```

7.4 Downstream Impact

```
# If your pipeline assumes ungrouped output:
result <- clinical_trial |>
  group_by(site, arm) |>
  summarise(mean_response = mean(response)) |>
  mutate(overall_mean = mean(mean_response))  # Computed by remaining group!

# Pre-dplyr 1.0.0: overall_mean is the grand mean (4 values)
# Post-dplyr 1.0.0: overall_mean is the site mean (2 values per site)
```

7.5 Defensive Solution

```
# Always specify .groups explicitly
summary_stats <- clinical_trial |>
  group_by(site, arm) |>
  summarise(mean_response = mean(response), .groups = "drop")

# Or ungroup explicitly after summarise
summary_stats <- clinical_trial |>
  group_by(site, arm) |>
  summarise(mean_response = mean(response)) |>
  ungroup()

# Use reframe() for multi-row returns
quantile_summary <- clinical_trial |>
  group_by(arm) |>
  reframe(
    quantile = c(0.25, 0.50, 0.75),
    value = quantile(response, c(0.25, 0.50, 0.75))
  )
```

8 Summary of Breaking Changes

Table 1 summarizes the breaking changes discussed in this white paper.

Table 1: Summary of Breaking Changes in R Packages

Package	Version	Change	Impact
Base R	4.0.0	stringsAsFactors default	Data types differ
dplyr	1.1.0	arrange() locale	Sort order changes
ggplot2	3.4.0	size to linewidth	Visual output differs
readr	1.2.0	Integer guessing removed	Column types differ
tidyr	1.0.0	gather/spread superseded	API migration needed
dplyr	1.0.0+	summarise() grouping	Grouping structure changes

9 The Case for `renv` and Docker

The examples above demonstrate that even well-maintained, widely-used packages introduce breaking changes that can:

1. **Silently alter results** without any error or warning
2. **Change statistical conclusions** when row ordering or grouping affects downstream analysis
3. **Break code entirely** when deprecated functions are removed
4. **Produce platform-dependent results** when locale or system settings influence behavior

9.1 Using `renv` for Package Version Management

`renv` creates a project-local library and records exact package versions in a lockfile:

```
# Initialize renv in a project
renv::init()

# Install packages (versions recorded in renv.lock)
renv::install("dplyr@1.0.10")
renv::install("ggplot2@3.3.6")

# Snapshot current state
renv::snapshot()

# Restore environment on another machine
renv::restore()
```

9.2 Combining with Docker

For complete reproducibility, combine `renv` with Docker to capture:

- R version
- System libraries
- Package versions
- Operating system environment

```
FROM rocker/r-ver:4.3.2

WORKDIR /project
COPY renv.lock renv.lock
```

```
RUN R -e "install.packages('renv')"
RUN R -e "renv::restore()"

COPY . .
CMD ["Rscript", "analysis.R"]
```

10 Conclusion

Breaking changes in R packages are inevitable as the ecosystem evolves. Defensive coding practices—such as explicit type declarations, named arguments, and locale specifications—can mitigate some risks. However, the only reliable way to ensure long-term reproducibility is to lock your computational environment using tools like `renv` and Docker.

The examples in this white paper demonstrate that changes affecting reproducibility are not hypothetical concerns but documented realities that have affected thousands of R users. Investing time in environment management is not overhead; it is an essential component of rigorous, reproducible research.

11 References

- Jumping Rivers. (2022). Forgotten features of R 4.0.0. https://www.jumpingrivers.com/blog/r-version-4-features/
- R Core Team. (2020). stringsAsFactors. https://developer.r-project.org/Blog/public/2020/02/16/stringsasfactors/
- Tidyverse Team. (2022). ggplot2 3.4.0. https://tidyverse.org/blog/2022/11/ggplot2-3-4-0/
- Tidyverse Team. (2022). Make your ggplot2 extension package understand the new linewidth aesthetic. https://tidyverse.org/blog/2022/08/ggplot2-3-4-0-size-to-linewidth/
- Tidyverse Team. (2019). tidyr 1.0.0. https://www.tidyverse.org/blog/2019/09/tidyr-1-0-0/
- dplyr changelog. https://dplyr.tidyverse.org/news/index.html
- readr documentation. https://readr.tidyverse.org/articles/column-types.html
- renv documentation. https://rstudio.github.io/renv/